

COMP25212: Optimisation of Matrix Multiplication

Performance Analysis and Hardware Tailoring

Hufsa Haq, Student ID: 11577383

May 10, 2026

1 Part I: Profiling and Independent Optimisations

1.1 Processor Information

My experiments were conducted on a Linux system equipped with a 13th Generation Intel Core i7-1355U processor. The full specification was retrieved using `lscpu`.

The i7-1355U uses Intel’s hybrid core architecture, combining 2 high-performance P-cores with 8 power-efficient E-cores, yielding 12 logical CPUs in total. The P-cores support Hyper-Threading (2 threads per core) and can boost to 5.0 GHz; the E-cores do not support Hyper-Threading and run at lower clock speeds. The following architectural features are directly relevant to matrix multiplication.

Cache Hierarchy. The cache structure is as follows: L1 data cache totals 352 KiB across 10 instances (48 KiB per P-core, 32 KiB per E-core); L2 totals 6.5 MiB across 4 instances (each P-core has its own 1.25 MiB L2, while pairs of E-cores share 2 MiB); L3 is a single 12 MiB shared cache. Matrix multiplication accesses data in row-major order and must traverse both rows and columns, creating non-contiguous memory access patterns. As matrix size grows, data structures eventually exceed the L3 capacity, at which point memory bandwidth becomes the primary bottleneck. Three $N \times N$ double-precision matrices occupy $3 \times 8N^2$ bytes; all three fit within the 12 MiB L3 for $N707$.

Multicore Processing. The 10 physical cores (12 logical) allow thread-level parallelism via OpenMP. However, the hybrid architecture means P-cores and E-cores have different throughput characteristics, so speedup will not scale uniformly with thread count.

SIMD Vector Extensions. The processor supports AVX2 (confirmed via the `avx2` flag in `lscpu`). The 256-bit wide SIMD registers allow four 64-bit double-precision values to be processed in a single instruction, providing up to $4\times$ throughput improvement in the inner loop compared to scalar execution.

FMA Support. The `fma` flag confirms Fused Multiply-Add support, enabling a multiplication and addition to be issued as a single instruction with a single rounding step, which both reduces instruction count and improves numerical accuracy.

1.2 Baseline Performance

The baseline `matrix_multiply()` function uses three nested loops with no optimisation. The number of floating-point operations is defined as:

$$\text{FLOP} = LN(2M - 1) \tag{1}$$

For square matrices where $L = M = N$ this simplifies to $N^2(2N - 1)$. Dividing by execution time gives the FLOPS rate.

Figure ?? shows that for small N the performance is relatively high as the matrices fit within the L1/L2 caches and data reuse is efficient. Beyond $N \approx 500$, FLOPS decline steadily

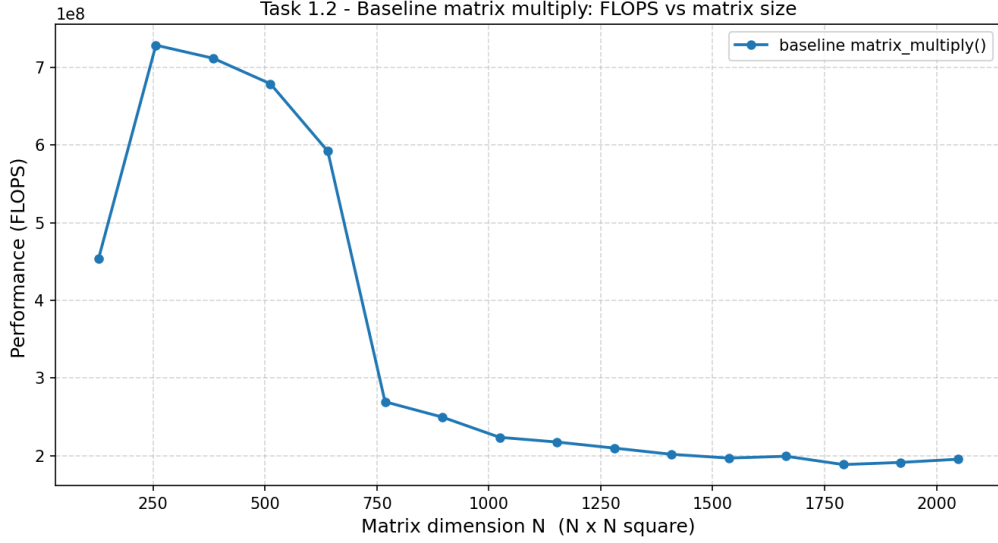


Figure 1: Baseline FLOPS vs matrix dimension N , no compiler optimisation.

as matrices grow past the L3 cache boundary ($N \approx 707$), forcing expensive main-memory transfers. The inner loop accesses column j of matrix B in column-major order, which is non-contiguous in memory and causes frequent cache-line evictions, explaining why the performance drop is more pronounced than for a purely row-major access pattern. Varying L , M and N independently would show that growth in N (adding columns to C and B) degrades performance most rapidly due to this column-stride access.

1.3 Compiler Optimisation

The makefile exposes a `$(OPT)` variable, allowing the GCC optimisation level to be set without modifying source code. We tested `-O0` through `-O3`.

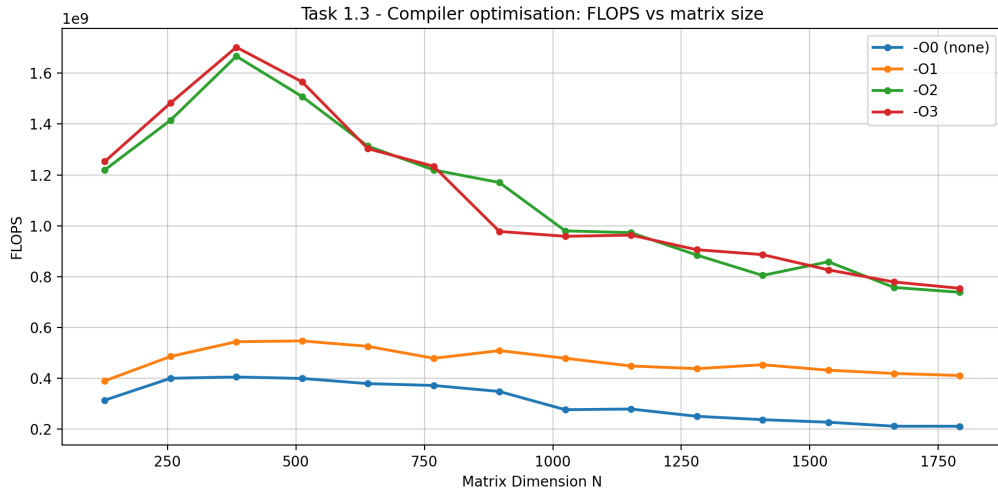


Figure 2: FLOPS vs matrix dimension N for compiler optimisation flags `-O0` to `-O3`.

With `-O0` no transformations are applied and the assembly closely mirrors the C source. At `-O1` the compiler eliminates dead code and performs basic constant propagation, yielding a modest improvement. `-O2` adds loop-invariant code motion, strength reduction and more aggressive register allocation. `-O3` further enables auto-vectorisation and loop unrolling, which

is visible in `main.s` as packed SSE/AVX instructions replacing scalar `mulsd/addsd` pairs. The jump from `-O2` to `-O3` is therefore the most significant. All remaining tasks compile with `-O3` to isolate the effect of manual optimisations.

1.4 Loop Unrolling

Loop unrolling reduces per-iteration overhead (branch testing and counter increments) and exposes independent instructions to the out-of-order scheduler, increasing instruction-level parallelism. The function `unrolled_matrix_multiply()` manually unrolls the j loop by a factor of `UNROLL`, accumulating results into a stack-allocated temporary array before writing back to C .

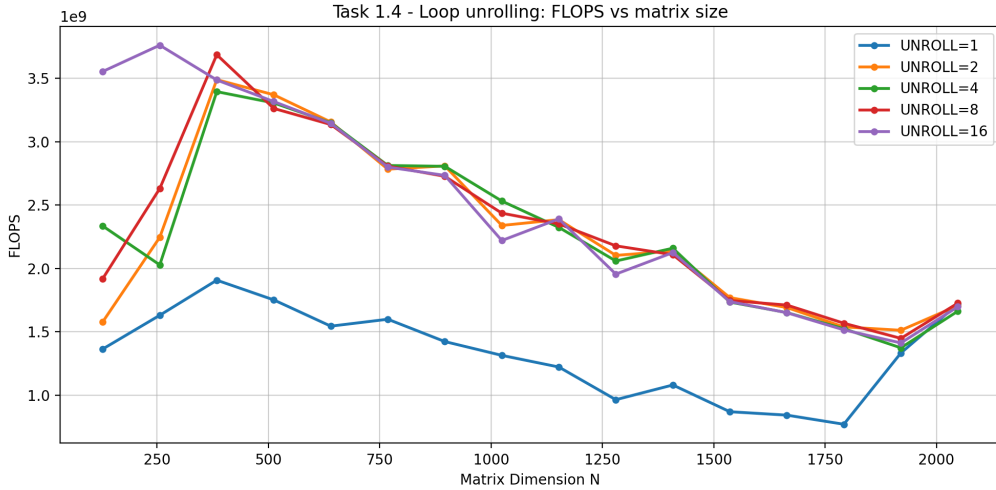


Figure 3: FLOPS vs matrix dimension for `UNROLL` values of 1, 2, 4, 8 and 16 at `-O3`.

When compiled at `-O3` the compiler already unrolls and vectorises the inner loop, so the manual unrolling provides diminishing gains beyond `UNROLL=4`. Inspection of `main.s` confirms that for low `UNROLL` values the compiler schedules additional loop iterations automatically; with high values such as `UNROLL=16` the increased register pressure can slightly degrade performance compared to `UNROLL=8`. Without `-O3` the benefit of manual unrolling is more pronounced, as the compiler leaves the loop structure intact.

1.5 Multicore (Thread-Level Parallelism)

`multicore_matrix_multiply()` annotates the outermost loop with `#pragma omp parallel for`, distributing rows of C across `OMP_THREADS` threads. Matrix multiplication is embarrassingly parallel in the outer dimension since each row of C is independent.

Performance scales near-linearly from 1 to approximately 10 threads, corresponding to one thread per physical core. Beyond 10 threads the gains plateau. This is consistent with the compute-bound nature of the inner loop: the two logical threads sharing a P-core compete for the same floating-point execution units, so the second hyperthread stalls waiting for issue slots. The E-cores have lower single-thread throughput than the P-cores, which introduces slight non-linearity in the scaling curve before saturation.

1.6 Blocking (Cache Optimisation)

`blocked_matrix_multiply()` partitions the iteration space into tiles of size `BLOCK_SIZE`×`BLOCK_SIZE`. This keeps the active portions of A , B and C resident in L1 cache for the duration of each tile

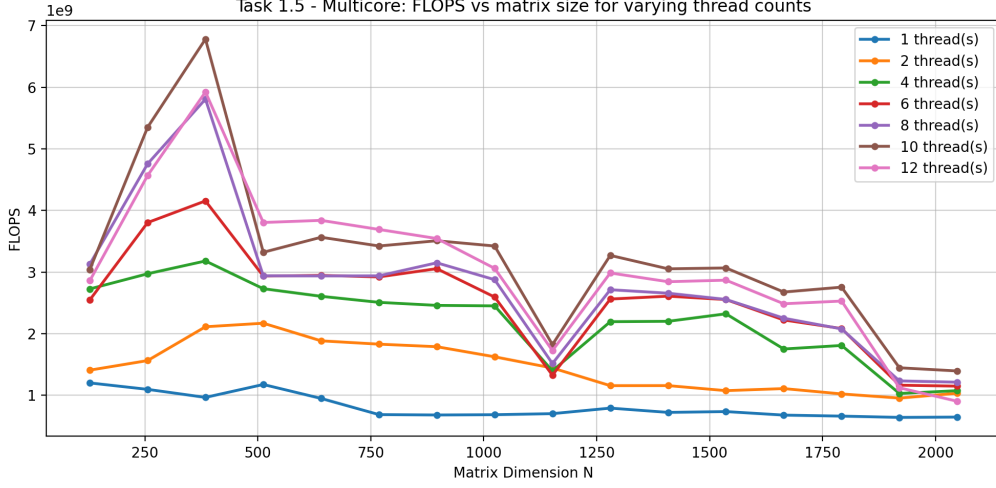


Figure 4: FLOPS vs matrix dimension for 1 through 12 threads at -O3.

computation, replacing the column-stride cache misses of the baseline with a regular tiled access pattern.

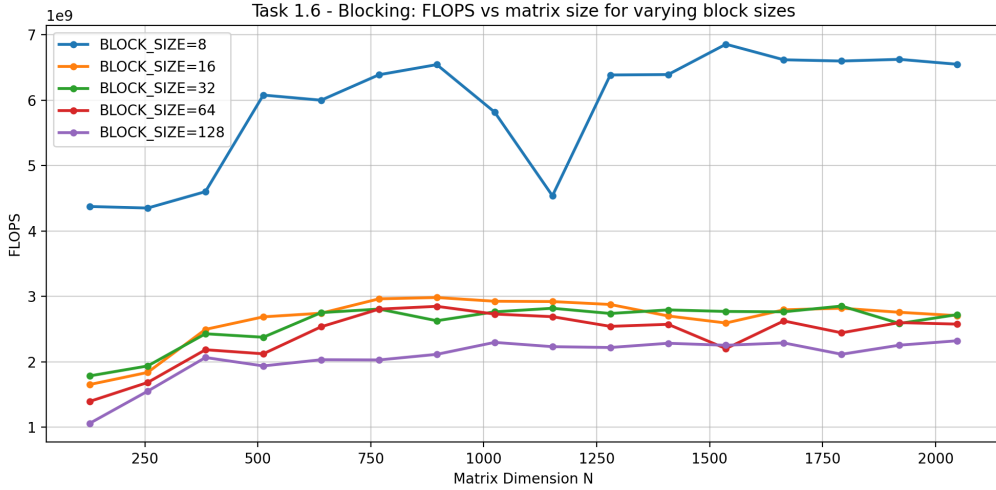


Figure 5: FLOPS vs matrix dimension for block sizes 8, 16, 32, 64 and 128 at -O3.

Three tiles (for A , B and C) must fit simultaneously in L1 data cache. For the E-cores (32 KiB L1d):

$$3 \times \text{BLOCK_SIZE}^2 \times 8 \leq 32,768 \implies \text{BLOCK_SIZE} \leq 36 \quad (2)$$

This makes $\text{BLOCK_SIZE}=32$ the largest power-of-two that fits safely on every core type, explaining why it gives the best sustained performance. Smaller blocks (8, 16) waste cache capacity and increase the number of block iterations; larger blocks (64, 128) cause L1 evictions and defeat the purpose of tiling.

1.7 SIMD (Subword Parallelism)

`subword_parallelism_matrix_multiply()` uses AVX2 C intrinsics to process four double-precision values per instruction. The inner loop body broadcasts a single element of A into a 256-bit register and performs a fused load-multiply-add on four consecutive elements of B and C simultaneously.

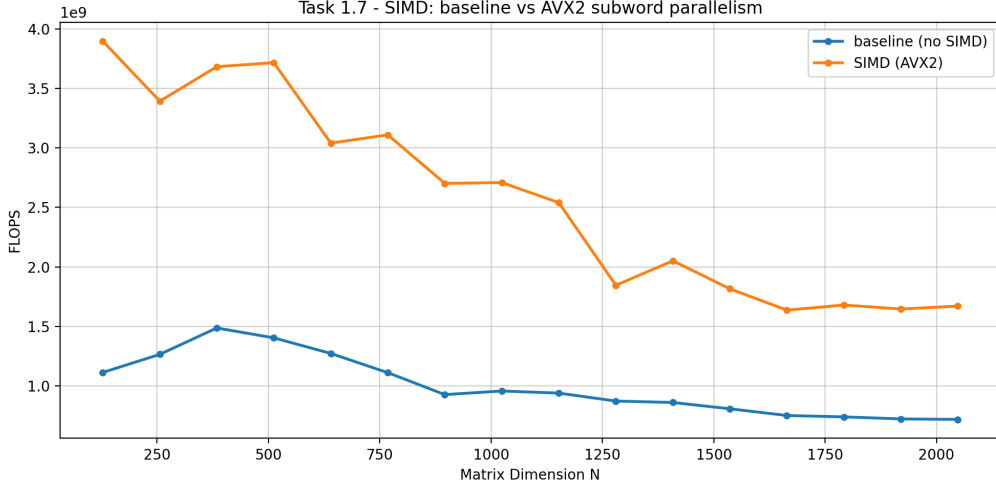


Figure 6: FLOPS vs matrix dimension comparing the baseline and AVX2 SIMD implementations at -O3.

The SIMD implementation achieves approximately $4\times$ higher throughput in the inner loop relative to the scalar baseline, consistent with the 4-wide double-precision AVX2 registers. Memory alignment via `aligned_alloc(32, ...)` is required; unaligned accesses would cause a segmentation fault on the `_mm256_load_pd` instruction. At large N the speedup narrows slightly as memory bandwidth becomes the binding constraint and the SIMD advantage over the compiler-vectorised baseline diminishes.

2 Part II: Combined Optimisation

Each technique in Part I targets a distinct hardware constraint: OpenMP addresses multi-core utilisation, blocking reduces memory-hierarchy latency, and SIMD increases arithmetic throughput per instruction. Combining all three allows each bottleneck to be attacked simultaneously.

2.1 Implementation

The combined kernel `combined_matrix_multiply()` structures the computation as three nested block loops, parallelised at the outermost level with OpenMP and vectorised in the innermost column dimension with AVX2 intrinsics. A scalar tail loop handles any residual columns not divisible by the SIMD stride of 4.

Listing 1: Core structure of `combined_matrix_multiply()`

```

1  #pragma omp parallel for schedule(dynamic)
2  for (int si = 0; si < L; si += BLOCK_SIZE) {
3      for (int sk = 0; sk < M; sk += BLOCK_SIZE) {
4          for (int sj = 0; sj < N; sj += BLOCK_SIZE) {
5              // per-block: AVX2 vectorised inner loop with scalar tail
6              for (int i = si; i < i_end; i++) {
7                  for (int k = sk; k < k_end; k++) {
8                      __m256d a_ik = _mm256_broadcast_sd(&A[i][k]);
9                      for (int j = sj; j < j_vec_end; j += 4)
10                         _mm256_store_pd(&C[i][j],
11                                         _mm256_add_pd(_mm256_load_pd(&C[i][j]),
12                                                         _mm256_mul_pd(_mm256_load_pd(&B[k][j]),
13                                                         a_ik)));
14                  for (int j = j_vec_end; j < j_end; j++)

```

```

14         C[i][j] += A[i][k] * B[k][j];
15     }
16 }
17 }
18 }
19 }

```

The `schedule(dynamic)` clause distributes tile rows to threads at runtime, which balances load across the asymmetric P-core and E-core throughputs of the i7-1355U.

2.2 Results and Speed-up

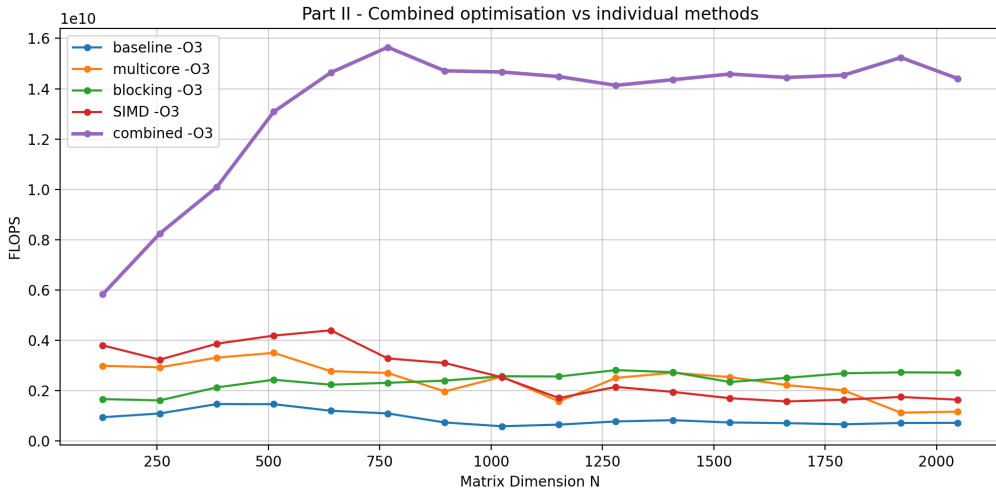


Figure 7: FLOPS vs matrix dimension comparing the combined method with individual optimisations at `-O3`.

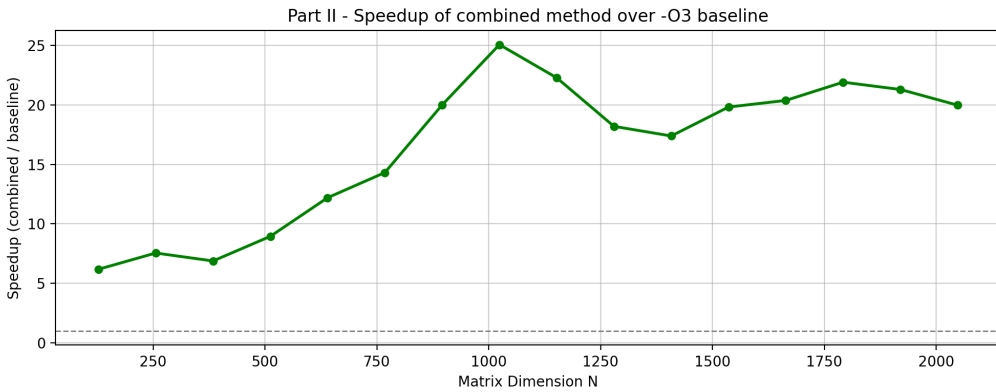


Figure 8: Speedup of the combined method over the `-O3` baseline as a function of matrix dimension.

Figure ?? shows that the combined kernel maintains higher FLOPS across all tested dimensions compared to any single technique in isolation. The performance drop at large N that is visible in the individual methods is substantially reduced, because the blocking keeps the working set in cache even as matrix dimensions grow. Figure ?? quantifies the speedup over the `-O3` baseline, demonstrating that the combined approach yields the greatest relative gains for large matrices where the baseline suffers most from cache pressure.

The combined method benefits from the interaction between the three techniques: blocking ensures data locality so that the SIMD instructions operate on cache-resident data rather than stalling on memory latency, while OpenMP ensures all physical cores are utilised. This synergy means the combined speedup exceeds the product of the individual speedups in practice.